Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-207БВ-25

Студент: Курочкин М.В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 23.09.26

Москва, 2026

Постановка задачи

Вариант 14.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должна создать для решения задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe).

Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Родительский процесс создает два дочерних процесса. Child1 и Child2 можно «соединить» между собой дополнительным каналом. Родительский и дочерний процесс должны быть представлены разными программами.

Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1. Процесс child1 и child2 производят работу над строками. Child2 пересылает результат своей работы родительскому процессу. Родительский процесс полученный результат выводит в стандартный поток вывода.

Child1 переводит строки в нижний регистр. Child2 убирает все задвоенные пробелы.

Общий метод и алгоритм решения

Использованные системные вызовы:

- pid_t fork(void); – создает дочерний процесс.
- int pipe(int *fd); – создает канал для передачи данных между процессами.
- int dup2(int oldfd, int newfd); — перенаправляет стандартные потоки ввода-вывода.
- ssize_t read(int fd, void *buf, size_t count); — читает данные из дескриптора.
- ssize_t write(int fd, const void *buf, size_t count); — записывает данные в дескриптор.
- int close(int fd); — закрывает файловый дескриптор.
- int execlp(const char *file, const char *arg, ...); — запускает новую программу, заменяя текущий процесс.
- pid_t wait(int *wstatus); — ожидает завершения дочернего процесса.
- ssize_t readlink(const char *path, char *buf, size_t bufsiz); — читает путь к исполняемому файлу.

Программа реализует схему межпроцессного взаимодействия с передачей сообщений «Конвейер». Выходной текст каждого процесса (stdout) передается непосредственно в качестве входного (stdin) в следующий процесс. Второй процесс запускается, когда первый еще выполняется, и они выполняются параллельно. Родительский процесс создает два дочерних процесса через fork(), настраивает каналы pipe и запускает дочерние процессы через exec. Обмен данными происходит по схеме parent -> pipe1 -> child1 -> pipe -> child2 -> pipe3 -> parent. Родительский процесс выполняет следующие действия:

1. Определяет путь к собственной директории.
2. Создает 3 канала посредством pipe(): от родительского процесса к первому дочернему процессу, от первого дочернего процесса ко второму, от второго дочернего процесса к родительскому.
3. Создает 2 процесса и настраивает их файловые дескрипторы с помощью close() и dup2(), после чего передает управление дочерним процессам, используя execlp().

4. Считывает read() из стандартного ввода введённые пользователем строки и передает их write() в дескриптор для чтения первого дочернего процесса, после чего считывает результат обработки из дескриптора для записи второго дочернего процесса. После этого выводит результат в терминал. После окончания обработки ввода закрывает оставшиеся концы каналов и ожидает окончания работы дочерних процессов, применяя wait().
5. Обрабатывает ошибки, возникающие при системных вызовах; сообщения об ошибках направляет в стандартный поток вывода ошибок stderr.

Первый дочерний процесс читает данные из стандартного ввода, который перенаправлен на канал от родительского процесса, приводит строки к нижнему регистру tolower() и записывает их в стандартный вывод, перенаправленный каналом на стандартный ввод второго процесса.

Второй дочерний процесс читает данные из стандартного ввода, удаляет лишние пробелы и направляет обработанные строки в стандартный вывод, перенаправленный каналом к родительскому процессу.

Код программы

parent.c

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <limits.h>

int main() {
	char path[PATH_MAX];
	ssize_t len = readlink("/proc/self/exe", path, sizeof(path) - 1);
	if (len == -1) {
		const char msg[] = "error: failed to read full program path\n";
		write(STDERR_FILENO, msg, sizeof(msg) - 1);
		exit(EXIT_FAILURE);
	} else {
		while(path[len] != '/') {
			len--;
		}
		path[len] = '\0';
	}

	int parent_to_child[2];
	if (pipe(parent_to_child) == -1) {
		const char msg[] = "error: failed to create a 'parent_to_child' pipe\n";
		write(STDERR_FILENO, msg, sizeof(msg) - 1);
		exit(EXIT_FAILURE);
	}

	int child_to_child[2];
	if (pipe(child_to_child) == -1) {
		const char msg[] = "error: failed to create a 'child_to_child' pipe\n";
		write(STDERR_FILENO, msg, sizeof(msg) - 1);
```

```
        exit(EXIT_FAILURE);
}

int child_to_parent[2];
if (pipe(child_to_parent) == -1) {
        const char msg[] = "error: failed to create a 'child_to_parent' pipe\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
}

const pid_t child_1 = fork();
if (child_1 == -1) {
        const char msg[] = "error: failed to spawn a new process\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
} else if (child_1 == 0) {
        close(parent_to_child[1]);
        close(child_to_child[0]);
        close(child_to_parent[0]);
        close(child_to_parent[1]);

        dup2(parent_to_child[0], STDIN_FILENO);
        close(parent_to_child[0]);

        dup2(child_to_child[1], STDOUT_FILENO);
        close(child_to_child[1]);

        char process_path[PATH_MAX + NAME_MAX + 2];
        snprintf(process_path, sizeof(process_path), "%s/child1", path);

        execlp(process_path, "child1", NULL);

        const char msg[] = "failed to exec into new executable image\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
}

const pid_t child_2 = fork();
if (child_2 == -1) {
        const char msg[] = "error: failed to spawn a new process\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
} else if (child_2 == 0) {
        close(parent_to_child[0]);
        close(parent_to_child[1]);
        close(child_to_child[1]);
        close(child_to_parent[0]);
```

```
            dup2(child_to_child[0], STDIN_FILENO);
            close(child_to_child[0]);

            dup2(child_to_parent[1], STDOUT_FILENO);
            close(child_to_parent[1]);

            char process_path[PATH_MAX + NAME_MAX + 2];
            snprintf(process_path, sizeof(process_path), "%s/child2", path);

            execlp(process_path, "child2", NULL);

            const char msg[] = "failed to exec into new executable image\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
    }

    close(parent_to_child[0]);
    close(child_to_child[0]);
    close(child_to_child[1]);
    close(child_to_parent[1]);

    char buffer[1024];
    ssize_t bytes = 0;
    ssize_t written = 0;

    while (bytes = read(STDIN_FILENO, buffer, sizeof(buffer))) {
            if (bytes < 0) {
                    const char msg[] = "error: failed to read from stdin\n";
                    write(STDERR_FILENO, msg, sizeof(msg) - 1);
                    exit(EXIT_FAILURE);
            } else if (buffer[0] == '\n') {
                    break;
            }

            written = write(parent_to_child[1], buffer, bytes);
            if (written != bytes) {
                    const char msg[] = "error: failed to write to parent_to_child\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
            }

            bytes = read(child_to_parent[0], buffer, sizeof(buffer));
            if (bytes < 0) {
    const char msg[] = "error: failed to read from child_to_parent\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}
```

```
                written = write(STDOUT_FILENO, buffer, bytes);
                if (written != bytes) {
                        const char msg[] = "error: failed to write to stdout\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
                }
        }

        close(parent_to_child[1]);
        close(child_to_parent[0]);

        wait(NULL);
        wait(NULL);

        exit(EXIT_SUCCESS);
}
```

child1.c

```
#include <unistd.h>
#include <stdlib.h>
#include <ctype.h>

int main() {
   char buffer[1024];
   ssize_t bytes = 0;
   ssize_t written = 0;

   while (bytes = read(STDIN_FILENO, buffer, sizeof(buffer))) {
      if (bytes < 0) {
         const char msg[] = "error: failed to read from stdin\n";
         write(STDERR_FILENO, msg, sizeof(msg) - 1);
         exit(EXIT_FAILURE);
      }

      for (ssize_t i = 0; i < bytes; i++) {
         buffer[i] = (char)tolower((unsigned char)buffer[i]);
      }

      written = write(STDOUT_FILENO, buffer, bytes);
      if (written != bytes) {
         const char msg[] = "error: failed to write to stdout\n";
         write(STDERR_FILENO, msg, sizeof(msg) - 1);
         exit(EXIT_FAILURE);
      }
   }

   exit(EXIT_SUCCESS);
}
```

child2.c

```
#include <unistd.h>
#include <stdlib.h>
#include <ctype.h>

int main() {
    char buffer[1024];
    ssize_t bytes = 0;
    ssize_t written = 0;

    while (bytes = read(STDIN_FILENO, buffer, sizeof(buffer))) {
        if (bytes < 0) {
            const char msg[] = "error: failed to read from stdin\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
        }

        int pos = 0;
        for (ssize_t i = 0; i < bytes; i++) {
            if (buffer[i] == '\n' || !isspace((unsigned char)buffer[i]) || (i > 0 && !isspace((unsigned
char)buffer[i - 1]))) {
                buffer[pos++] = buffer[i];
            }
        }

        written = write(STDOUT_FILENO, buffer, pos);
        if (written != pos) {
            const char msg[] = "error: failed to write to stdout\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
        }
    }

    exit(EXIT_SUCCESS);
}
```

Протокол работы программы

```
mikhail@mikhail-VMware-Virtual-Platform:~/Desktop$ cat tests
HELLO WORLD
HELLO    WORLD
   SPACES    AT    START
SPACES AT END
HeLLo WoRLd
HELLO, WORLD! 123
MULTIPLE   SPACES   BETWEEN   WORDS
UPPERCASE
mIxEd CaSe
A B C D E
mikhail@mikhail-VMware-Virtual-Platform:~/Desktop$ ./parent < tests
```

hello world
hello world
spaces at start
spaces at end
hello world
hello, world! 123
multiple spaces between words
uppercase
mixed case
a b c d e
mikhail@mikhail-VMware-Virtual-Platform:~/Desktop$

Вывод

В ходе лабораторной работы я приобрел практические навыки в управлении процессами в ОС и обеспечении обмена данными между процессами посредством каналов. Составил и отладил программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в операционной системе Linux (Ubuntu). В результате работы программа (основной процесс) создала для решения задачи 3 дочерних процесса. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Программа обрабатывает системные ошибки, которые могут возникнуть в результате работы.